

ASYNCHRONOUS JAVASCRIPT

asynchronous functions

provided by the JavaScript environment, *whose execution does not block the main thread*.

JavaScript environment delegates asynchronous functions to background processes (in Web APIs) and upon completion, a callback function is triggered to process the result.

callback function

is a regular function, passed as an argument to asynchronous function. *All callbacks are executed after the top-level synchronous code has finished.*

custom asynchronous functions

any function that calls a asynchronous function must itself be asynchronous, using a callback to deliver its result.

An example

The window object of the DOM has a method called **setTimeout()** that you can use to run a function (callback) after a specific amount of time. The method accepts two arguments: 1. The function to execute after the timeout 2. The amount of time to wait in milliseconds.

The function `getData()` calls the (native-) asynchronous function `setTimeout()`, must itself be asynchronous, does not block the execution of the function `nextTask()`.

The callback `logResult(result)` processes the result of asynchronous operation.

```
function logResult(data) { // callback function
  console.log(data)
}

function getData(callback, URL) { // Simulating fetching user data from an API
  setTimeout( function() {
    const response = 'Hello! from ' + URL
    callback(response)      // callback processes result of asynchronous operation
  }, 1000)
}

function nextTask() {
  console.log("End")
}

console.log("Start")
getData(logResult, "myhost/api/data")
nextTask()
```

When you run the code above, you will see this output:

```
Start
End
Hello! from myhost/api/data
```

JavaScript is a non-blocking, synchronous programming language.

Non-blocking means that JavaScript doesn't wait until a code finished running before running the next one. Synchronous means sequential, and we know this because JavaScript runs code line by line from the top to the bottom.

The 'End' string got printed ahead of the 'Hello! ...' string because JavaScript starts the `getData()` method, then continue code execution to the next line.

callback hell

Callback hell typically occurs when dealing with a series of asynchronous operations, such as making API requests, where one operation depends on the result of another or previous one. The callback itself has to call functions that accept a callback. As a result, they are nested. An example of what callback hell might look like:

```
asyncOperation1(function(result1) { // callback level 1
  asyncOperation2(result1, function(result2) { // callback level 2
    asyncOperation3(result2, function(result3) { // callback level 3
      // ...
    })
  })
})
```

Promise

A promise represents the eventual completion or failure of an asynchronous operation, allows you to chain operations using methods like `then()` and `catch()`.

Promise simplifies error handling, eliminates nested callback functions, and improves code readability.

implement asynchronous code/function using promise and callback

Create a Promise ---

A promise represents the eventual completion or failure of an asynchronous operation

```
function fetchData(URL) {

  let promise = new Promise(function(resolve, reject) { // executor function
    // asynchronous operation produces result or error

    // let success = true; // or false
    // let result = ...
    // let error = ...

    if (success) {
      resolve(result);
    } else {
      reject(error);
    }
  });
  return promise;
}
```

The constructor function takes a function as an argument called the executor function. The executor function takes two arguments, `resolve` and `reject`. Your logic goes inside the executor function that runs automatically when a new Promise is created.

The returned promise object should be capable of informing when the execution has been started (Pending), completed (resolved) or returned with error (rejected).

Consume a Promise ---

Once you have a reference, you can attach callback function by using the `.then` and `.catch` methods.

Once the promise is resolved, the `.then` callback method will be called with the resolved value.

And if the promise is rejected, the `.catch` method will be called with an error message.

```
fetchData(URL)
  .then(function(result) { // callback function // This runs if the promise is fulfilled
    console.log(result);
  })
  .catch(function(error) { // This runs if the promise is rejected
    console.log(error);
  });
```

A typical promise implementation would look as follows

When creating a new Promise object, we need to pass a callback function that will be called immediately with two arguments: the `resolve()` and `reject()` functions

Depending on the result of the Promise, either the `resolve()` or the `reject()` function will be called to end the pending state.

To handle the Promise object, you need to chain the function call with `then()` and `catch()` functions as shown below:

```
let p = new Promise((resolve, reject) => {
  let isTrue = true;
  if (isTrue) {
    resolve('Success');
  } else {
    reject('Error');
  }
});

p
  .then(message => console.log(`Promise resolved: ${message}`))
  .catch(message => console.log(`Promise rejected: ${message}`));

console.log('start');
console.log('end');
```

Output:

```
start
end
Promise resolved: Success
```

The `resolve()` function corresponds to the `then()` function, while `reject()` corresponds to the `catch()` function. You can change the `isTrue` value to `false` to test this.

Promises Chain

```
console.log('Start');
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(response => response.json())
  .then(data => console.log(data))
  console.log('End');
```

// output =>

```
console.log('Start');
fetch('https://jsonplaceholder.typicode.com/todos/1')
  .then(function(response) {
    return response.json();
  })
  .then(function(data) {
    console.log(data);
    // return true;
  })
  console.log('End');
```

// output =>

Chaining Promises using `async/await` syntax

`async/await` provides a way to write asynchronous code in a synchronous style.

`async/await`, it's just syntactic sugar for Promises. In other words, any existing Promises can be converted to use `Async/Await` syntax instead.

What `async/await` allows us to do is instead of having to access the results of our Promises inside a `then` method, we're able to assign the result of an asynchronous operation directly to a variable. Even though this code looks synchronous, it behaves asynchronously.

`await` keyword is used inside a **`async`** function (function that contains asynchronous code), and therefore we have to put the `async` keyword before the function definition.

```
async function getData(URL) {  
  try {  
    const response = await fetch(URL);  
    const data = await response.json();  
    console.log(data);  
  } catch (error) {  
    console.log("Error:", error);  
  }  
}  
  
console.log('Start');  
getData('https://jsonplaceholder.typicode.com/posts/1');  
console.log('End');
```

// output

Start

End

(4) {userId: 1, id: 1, title: "delectus ...}